In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

print("All libraries imported successfully! ✅")
```

```
Matplotlib is building the font cache; this may take a moment.
All libraries imported successfully! ✅
```

In [2]:
```
df = pd.read_csv('https://raw.githubusercontent.com/IBM/telco-customer-churn-on-

# Show first 5 rows
df.head()
```

Out[2]:

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	Mul
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

5 rows × 21 columns

In [3]:
```
print("Shape of data (rows, columns):", df.shape)
print("\nColumn names:\n", df.columns.tolist())
print("\nData types of each column:\n", df.dtypes)
print("\nMissing values in each column:\n", df.isnull().sum())
```

```
Shape of data (rows, columns): (7043, 21)

Column names:
 ['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'Ph
oneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBacku
p', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contrac
t', 'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Chur
n']

Data types of each column:
 customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges        object
Churn               object
dtype: object

Missing values in each column:
 customerID         0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

In [4]:
```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

# Fill the 11 missing values with the median
```

```
df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

print("✅ TotalCharges column is now fixed and ready!")
print("Missing values left in TotalCharges:", df['TotalCharges'].isnull().sum())
```

```
✅ TotalCharges column is now fixed and ready!
Missing values left in TotalCharges: 0
```

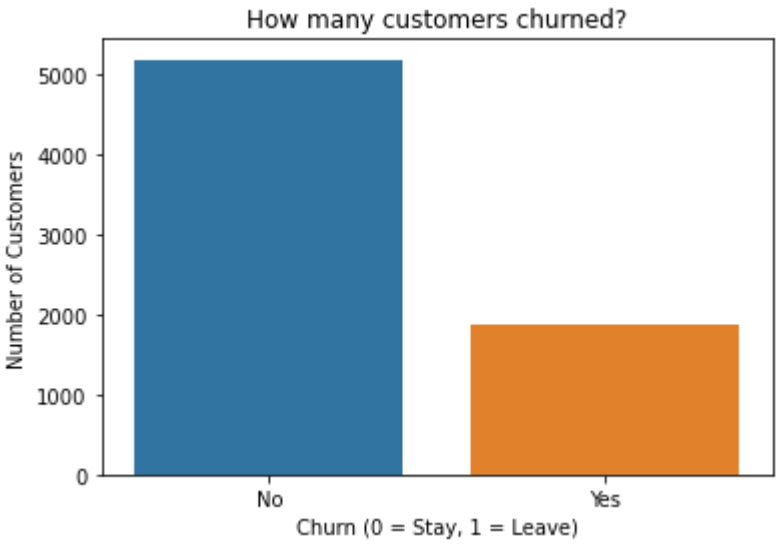
In [5]:

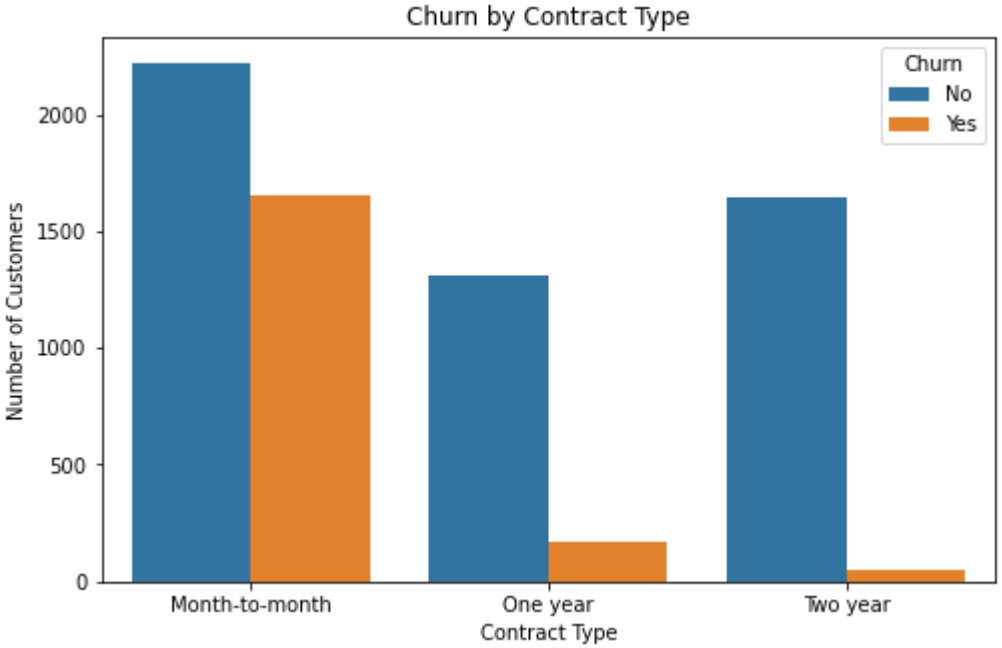
```
import matplotlib.pyplot as plt
import seaborn as sns

# Plot 1: How many customers churned?
plt.figure(figsize=(6,4))
sns.countplot(x='Churn', data=df)
plt.title('How many customers churned?')
plt.xlabel('Churn (0 = Stay, 1 = Leave)')
plt.ylabel('Number of Customers')
plt.show()

# Plot 2: Churn by Contract Type
plt.figure(figsize=(8,5))
sns.countplot(x='Contract', hue='Churn', data=df)
plt.title('Churn by Contract Type')
plt.xlabel('Contract Type')
plt.ylabel('Number of Customers')
plt.legend(title='Churn')
plt.show()

print("✅ Two beautiful plots created successfully!")
```





✅ Two beautiful plots created successfully!

In [6]:
```
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})
df.drop('customerID', axis=1, inplace=True)
print("✅ Churn is now 0 and 1!")
print("customerID column removed!")
print("Current shape of data:", df.shape)
```

```
✅ Churn is now 0 and 1!
customerID column removed!
Current shape of data: (7043, 20)
```

In [7]:
```
cat_cols = df.select_dtypes(include='object').columns
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)
print("✅ All columns are now numbers!")
print("New shape of data:", df.shape)
print("First 5 column names now:", df.columns[:5].tolist())
```

```
✅ All columns are now numbers!
New shape of data: (7043, 31)
First 5 column names now: ['SeniorCitizen', 'tenure', 'MonthlyCharges', 'TotalCha
rges', 'Churn']
```

In [8]:
```
from sklearn.model_selection import train_test_split
X = df.drop('Churn', axis=1)
y = df['Churn']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_

print("✅ Data split successfully!")
print("Training data size:", X_train.shape)
print("Testing data size:", X_test.shape)
```

```
✅ Data split successfully!
Training data size: (5634, 30)
Testing data size: (1409, 30)
```

In [9]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print("✅ Model trained successfully!")
print("Accuracy on test data:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
✅ Model trained successfully!
Accuracy on test data: 0.8176011355571328

Classification Report:
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1036
           1       0.68      0.59      0.63       373

    accuracy                           0.82      1409
   macro avg       0.77      0.75      0.76      1409
weighted avg       0.81      0.82      0.81      1409
```

In [10]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
tree_model = DecisionTreeClassifier(max_depth=5, random_state=42)
tree_model.fit(X_train, y_train)
y_pred_tree = tree_model.predict(X_test)
print("✅ Decision Tree trained successfully!")
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_tree))
print("\nClassification Report:\n", classification_report(y_test, y_pred_tree))
```

```
✅ Decision Tree trained successfully!
Decision Tree Accuracy: 0.8062455642299503

Classification Report:
              precision    recall  f1-score   support

           0       0.83      0.93      0.88      1036
           1       0.70      0.46      0.56       373

    accuracy                           0.81      1409
   macro avg       0.77      0.70      0.72      1409
weighted avg       0.80      0.81      0.79      1409
```

In [ ]:
```

```